Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (AI & ML - EC, CSE, CSE (DS), IT, ECE & Civil)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 08933-225083/84/87/225073, Website: www.anits.edu.in

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2025-27	Semester:	III/IV Sem-I
Name of the Student:	B. Sowmya Gitika	Roll Number:	322126502023
Name of the Lab:	Full Stack lab	Timings:	

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (10 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (3 Marks)	Programming Knowledge (3 Marks)	Results (2 Marks)	Viva Voce (2Marks)			
1/7	1					14		
8/7	2					14		
15/7	3					14		
22/7	4					15		
29/7	5					15		
5/8	6					15		
12/8	7					14		
19/8	8					15		
2/9	9					13		
~~8/9~~	~~10~~							

Name of the Faculty: Ch. Sandhya

Faculty Signature: Ch. Sandhya

01/07/2026

Introduction to HTML to CSS

→ HTML (Hypertext Markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpages. It Using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices

css can be applied using inline, internal or external Stylesheets.

→ HTML and CSS work together to develop webpage, where Html provides the structure and content. while css provides the styling. layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

* Difference between MERN stack and Full stack

→ MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN Stands for Mongo db, Express Js, React JS, Node Js. MongoDB is used for database management, Express.js handles the backend framework and APIs. React.js is used to develop frontend user interface and Node.js provides the Server Side runtime environment.

→ Full-stack Development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing Server-side logic, Managing databases.

08/07/2026

HTML Components

1. HTML Tags: - Define the structure and elements of a web page...
2. Headings : - Used to display titles and headings using <h1> to <h6>
3. Paragraphs: - Used to display text using the <p> tag.
4. Links - Used to navigate between webpages using <a> tag.
5. Images - Used to display images using <img> tag.
6. Lists - Used to create ordered and unordered lists using <ol>, <ul>, <li>.
7. Tables - Used to organize data in rows and columns.
8. Forms - Used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div & span - Used to group and organize webpage content.
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provide meaningful webpage structure.

CSS Components

1. Selectors - Select HTML elements to apply styles
2. Properties - Define the visual characteristics of elements.
3. Values - Specify the setting of a css property
4. Colors - Define text, background, border colors.
5. Fonts - Control font family, size, style and weight.
6. Backgrounds - Set background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - Controls the space outside an element.
9. Padding - Controls the space inside element.
10. Layout - Controls the arrangement of elements using flexbox, grid and positioning.

HTML VS HTML5

HTML	HTML5
→ Older version of HTML	→ latest major version of HTML.
→ Limited multimedia support.	→ Supports audio and video.
→ Uses external plugins for multimedia.	→ No external plugins needed.
→ Has fewer semantic tags.	→ Provides semantic tags.
→ No <canvas> element.	→ Supports <canvas>
→ Limited web storage.	→ Supports local Storage.
→ Complex DOCTYPE declaration.	→ Simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS.
Basic styling features.	Provides advanced styling features.
Limited animations.	Supports animations and transitions.
No flexible layout systems.	Supports flexbox and grid.
Limited visual effects.	Supports shadows, gradients and effects.
No media queries.	Supports Media queries.
Uses a single specification.	Divided into different modules.

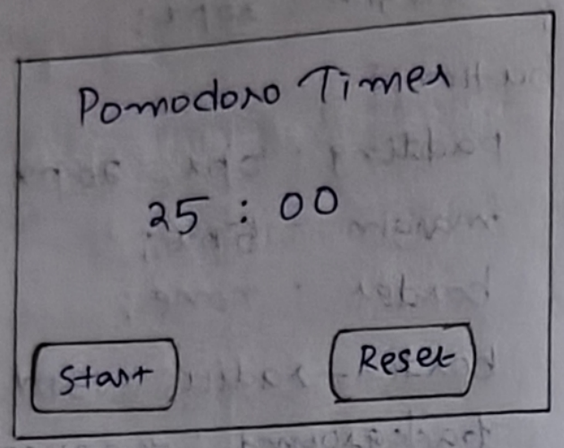
15/02/2026

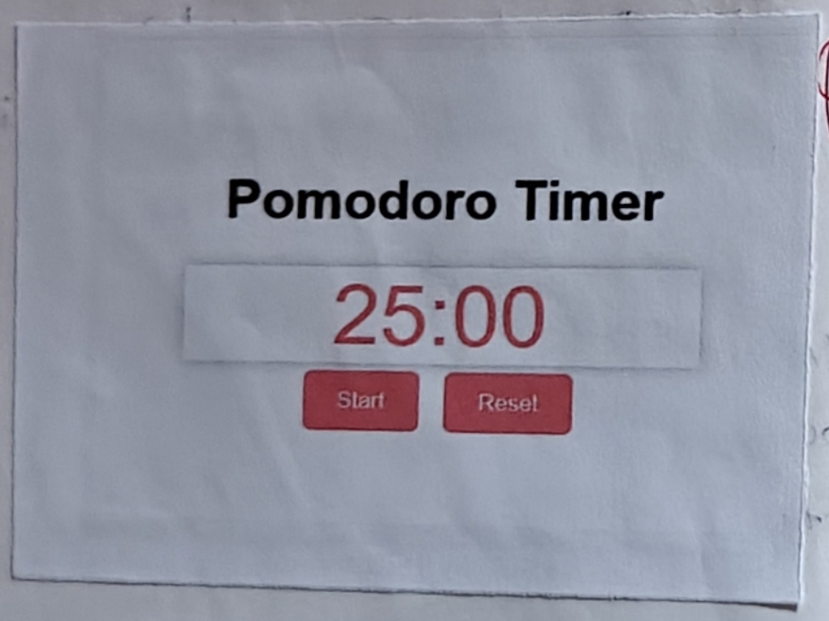
Design a static webpage using HTML and CSS

Pomodoro Timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
  <title> Pomodoro Timer </title>
  <style>
    body {
      font-family : Arial;
      background : #f5f5f5 ;
      text-align : center;
      padding : 50 px;
    }
.timer {
    width : 300px ;
    margin : auto ;
    padding : 30px ;
    background : white ;
    border-radius : 15px ;
    box-shadow : 0 0 10px gray;
}
h1 {
   color : #e74c3c;
}
.time {
    font-size : 50px ;
    margin : 20px ;
}
button {
    padding : 10px, 20 px ;
    margin : 5px ;
    border : none ;
    border-radius : 5px ;
    background : #e74c3c ;
  } color : white ;
  </style>
</head>
<body>
  <div class = "Timer">
    <h1> Pomodoro Timer </h1>
    <div class = "time"> 25:00 </div>
    <button> Start </button>
    <button> Reset </button>
  </div>
</body>
</html>
```

Output :





22/07/2026

i) Design an HTML code using audio tags.

Code :

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My favorite Song </h2>
    <audio controls>
        <source src="song.mp3" type="audio/mpeg">
        Your browser does not support audio tag
    </audio>
</body>
</html>
```

O/P : Displays an audio player with play, pause and volume controls.

Steps :

Create a new HTML file.

Add the basic HTML structure.

Add the heading using <h2> tag.

Insert the <audio> tag with controls attributes.

Specify the audio file using <source> tag.
Save and open the HTML file in a web brow
-ser.
Play the audio using displayed controls.

ii) Design a HTML code using video tags.

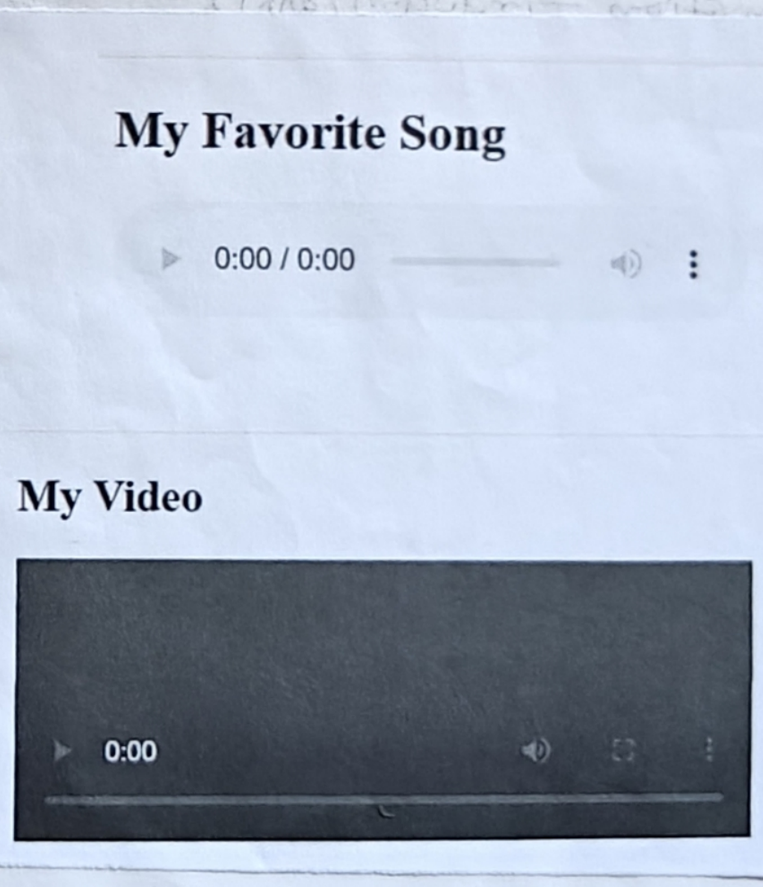
Code:

```
<!DOCTYPE html>
<html>
<head>
   <title> Video Example </title>
</head>
<body>
   <h2> My video </h2>
   <video width = "400" controls >
     <source src = "video.mp4" type = "video/mp4">
   </video>
</body>
</html>
```

O/p: Displays a video player with play, pause, volume and full screen controls.

Steps

Create a new HTML file.
Add a basic HTML structure.
Add a heading using the <h2> tag.
Insert the <video> tag with controls attrib
-ute.
Specify the video file using <source> tag.
Save the file and open it in web browser.
Play the video using displayed controls.



Write a program using javascript (arrays, functions).

code:

```
let numbers = [12,45,8,23,56,19,34,67,90,];
function displayArray(arr){
    console.log("Array Elements:");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findSum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

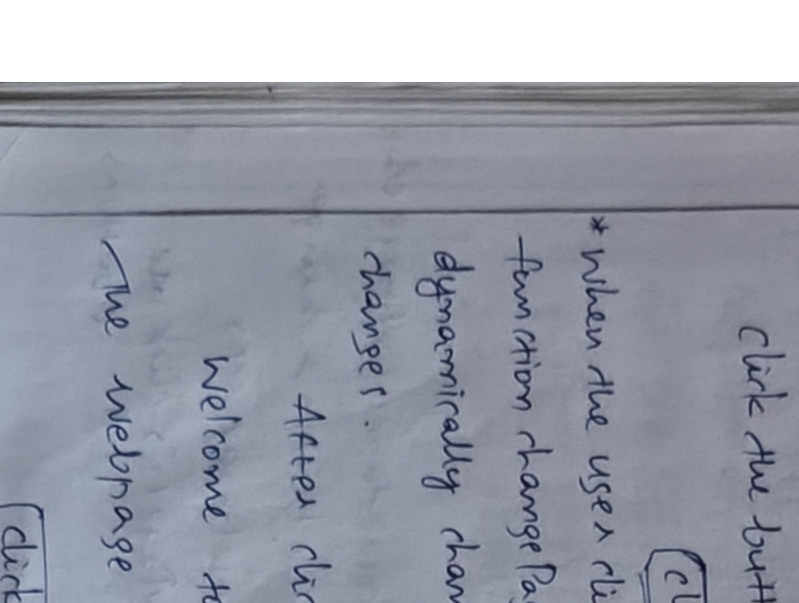
Experiment-4.

12. Develop dynamic webpage javascript program

```
<!DOCTYPE html>
<html>
<head>
  <title> Dynamic webpage </title>
  <style>
    body {
      text-align : center;
      font-family : Arial;
      margin-top : 100px;
    }
    button {
      padding : 10px 20px;
      font-size : 10px;
      cursor : pointer;
    }
  </style>
</head>
<body>
  <h1 id="heading"> welcome to my webpage </h1>
  <p id="message"> click the button to change
                   this webpage </p>
  <button onclick="changePage()"> click me
                                  </button>
```

```
<script>
function changePage() {
    document.getElementById("heading").
    innerHTML = "Welcome to JavaScript!";
    document.getElementById("message").
    innerHTML = "The webpage has been
        changed dynamically";
    document.body.style.backgroundColor =
        "lightblue";
}
</script>
</body>
</html>
```

Output



Welcome to my webpage

Click the button to change this webpage

Click Me

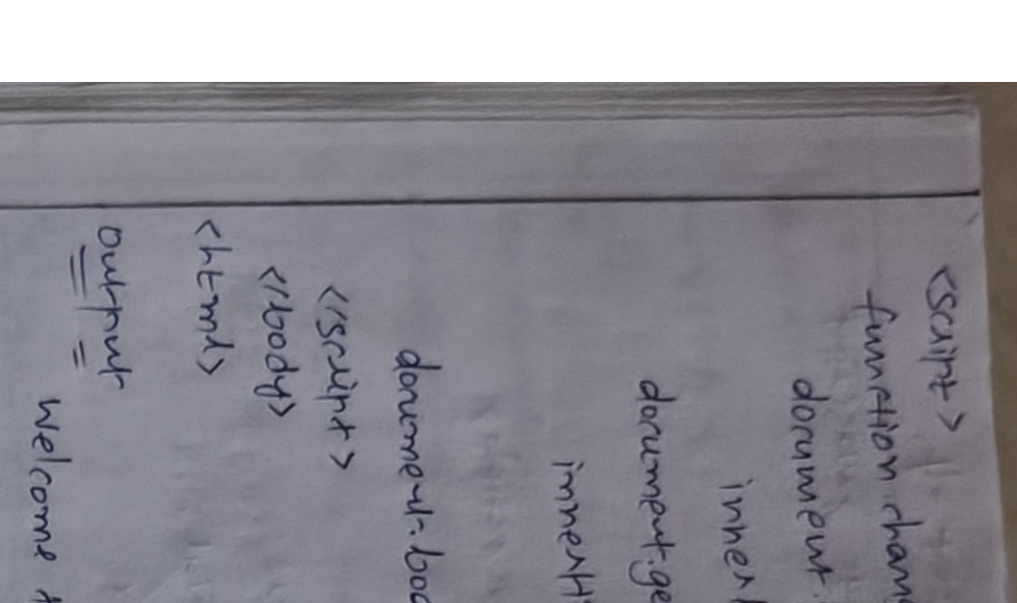
* When the user clicks "Click Me" the javascript function changePage() is called, then the heading dynamically changes and background also changes.

After clicking the button

Welcome to Javascript

The webpage has been changed dynamically

Click Me



Experiment - 5

* Implement Modules in Node.Js.

In Node.Js, you can implement modules using two distinct systems, CommonJs (Js), which is traditional default and Es modules (ESM), the modern javascript standard.

1. The CommonJs Method (default)

CommonJs uses Module.exports to share code & require() to import it. This system loads modules synchronously.

a) Create the Module (Math.js)

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
        add,
        subtract
};
```

b) Use the Module (app.js)

```
const math = require('./math');
console.log (Math.add (10, 5));
console.log (math.subtract (10,5));
```

output: 15
5

2. The Es modules Method (Modern Standard)

Es module use export to share code & import to use it make Node.js recognize Es modules, you must either use the mjs file extension (or) add "type": "module" to your package.json file.

a. Create the module (calculator.js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorNaive = "supercalc";
export default calculatorName;
```

b. Use the Module (Main.Js)

```
import calculatorName {multiply, divide}
        from "./calculator.js";
console.log (calculatorName);
console.log (calculatorName);
console.log (multiply(4,2));
console.log (divide (4,2));
```

3. Using built in core Modules:

Node.js also equipped with dozens built in core modules like fs for file systems (or) path for file routing) that don't need insta-llation.

we will load them by modules names:

```
// common JS style
const fs = require("fs");
// fs modules style
import fs from "fs";
```

Here are some of the most commonly and ones;

1. fs = file system operations
2. http = HTTP server and client
3. path = File path utilities
4. os = Operating System utilities.
5. Events = Event handling
6. Util = Utility functions
7. Stream = Stream handling
8. Crypto = Cryptographic functions
9. url - URL passing.
10. querystring - URL query string handling

Ex: Simple HTTP server

```
http.createServer((req,res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end("Hello world!");
}); listen(8080);
```

output : Hello world!

Experiment - 6

6. Develop mini application using express and Node.Js.

* Simple mini application using nodeJs + expre -ss Js.

Student Management System

1. Add a student
2. View all student
3. Delete a student
4. Uses express.js routes
5. Simple HTML interface.

project structure

```
student-app/
├── server.js
└── public
    └── index.html
```

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/student", (req, res) => {
        res.json(students);
});
```

```
// Add
app.post("/students", (req, res) => {
    const student = {
        id : student.length+1;
        name : req.body.name;
        course : req.body.course;
    };
    students.push(student);
    res.json({message : 'student added
                        successfully"
                student : students});
});
// Delete
app.delete('/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    student = students.filter(students =>
                student.id !==id);
    res.json({
        message : 'student delete successfully"});
});
// start server
app.listen (PORT, () => {
    console.log ('server running at
                http://localhost:${PORT}');
});
```

Public / index.html:

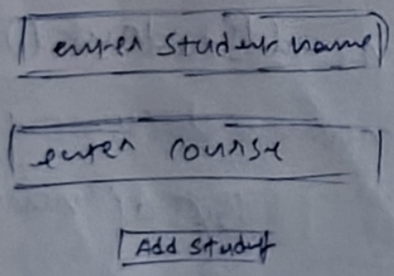
```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type="text" id = "name"
     placeholder = "enter student name">
    <input type = "text" id = "course"
     placeholder = " enter course" />
    <button onclick = "addStudent()"> Add studen
     t </button>
    <h2> Student list </h2>
    <ul id = "Studentlist "> </ul>
    <script>
     function addStudent() {
        let name = document.getElement
                    ById("name").value;

     fetch("/student", {
            method = "POST",
            headers = {
```

```
        "content-type": "application/json"
      },
      body: JSON.stringify ({
          name : name;
          course : course
      })
})
.then(response => response.json())
.then(data => {
          alert(data.message);
          displayStudent();
  });
}
displayStudent();
</script>
</body>
</html>
```

output

Student Management System

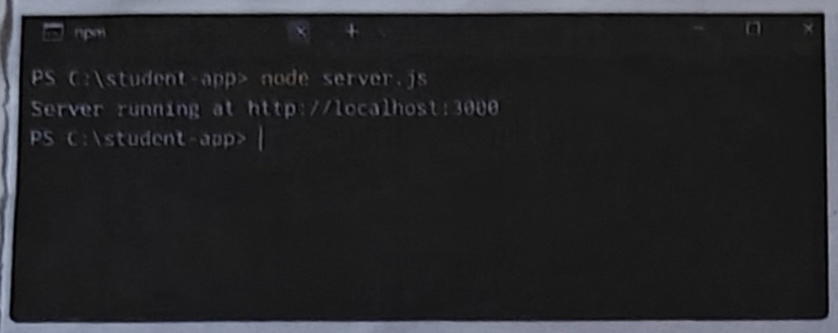


student list

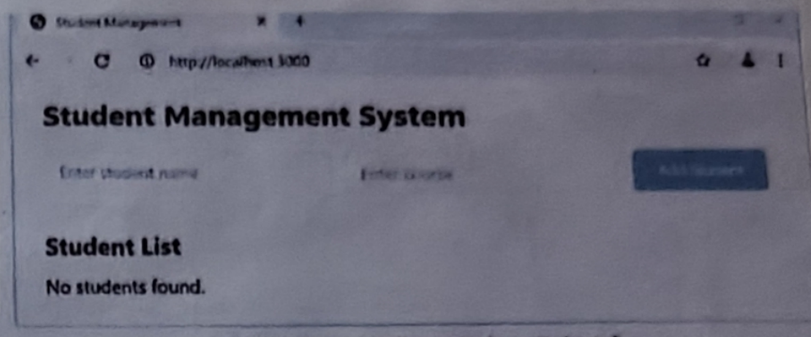
1. Girish - CSM (delete)
2. Hari - CSE (delete)

* The express.js server is successfully started at http://localhost:3000

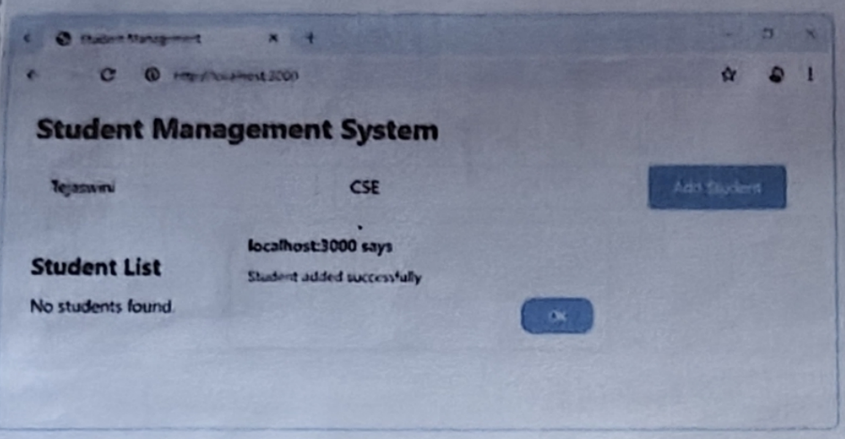
The browser display the student management system with fields.



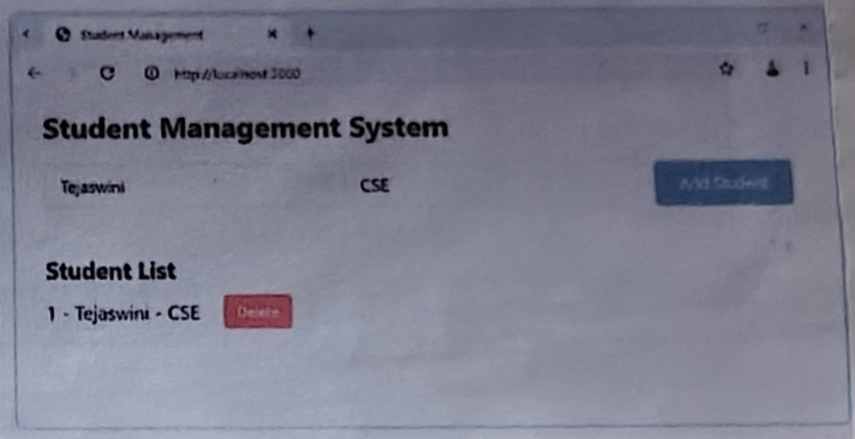
1. Terminal Output (after running `node server.js`)



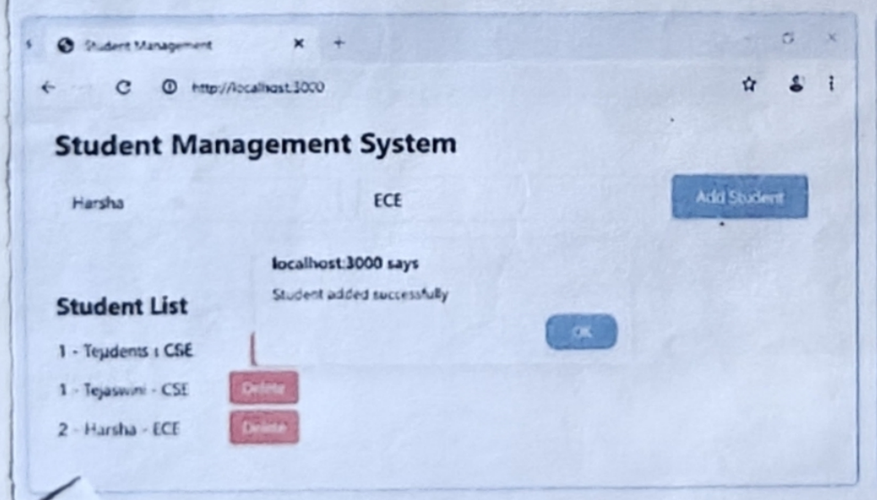
2. Initial Output in Browser (no students)



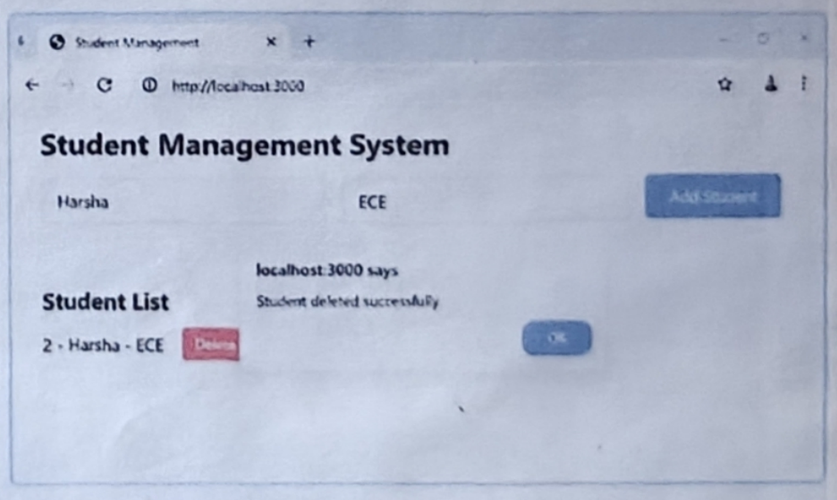
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment -7

7. Create a functional component and class component using react.

Aim: To develop a react application using functional components and class components to display student details manage state, handle button clicks and dynamically update data on a webpage.

*Functional Component

A functional component is a javascript function that returns JSX to display the user interface. It uses React hooks such as useState to manage state and update the webpage dynamically.

Program:

```
import React, {useState} from 'react';
function StudentFunction() {
  const [name, setName] = useState("Girish");
  const [course, setCourse] = useState("CSE");
  const [count, setCount] = useState(0);
  const changeName = () => {
      setName("Girish");
  };
  const changeCourse = () => {
      setCourse('CSE');
  };
  const incrementCount = () => {
      setCount(count+1);
  };
  return(
   <div style={{
       border: "2px solid blue",
       padding: "20px",
       margin: "20px",
       borderRadius: "10px",
       textAlign: "center"
   }}>
   <h1> Student Details </h1>
   <h2> Functional Component </h2>
   <p> <b> Name: </b> {name} </p>
   <p> <b> Course: </b> {course} </p>
   <p> <b> Button clicks : </b> {count} </p>
   <button onClick={changeName}>
       change Name ←
   </button>
   <button onClick={changeCourse}>
       change course
   </button>
```

```
incrementCount = () => {
    this.setState((prevState) => {
        count: prevState.count + 1;
    });
};
```

```
render() {
    return (
        <div style={{
            border: "2px solid green",
            padding: "20px",
            margin: "20px",
            borderRadius: "10px",
            textAlign: "center"
        }}>
            <h1> Student Details </h1>
            <h2> Class Components </h2>
            <p> <b> Name: </b> {this.state.name} </p>
            <p> <b> Course: </b> {this.state.course} </p>
            <p> <b> Button clicks: </b> {this.state.count} </p>
            <button onClick={this.changeName}>
                Change Name
            </button>
            <button onClick={this.changeCourse}>
                Change Course
            </button>
            <button onClick={this.incrementCount}>
                Increment
            </button>
```

React Components

Student Details

Functional Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Experiment - 8

8. Implement http methods using JSON Server

Aim: To implement HTTP methods such as GET, POST, PUT, DELETE using JSON Server to perform CRUD (Create, Read, Update, and Delete) operations on Student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file it allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between client and a server.

* GET - Retrieves data from the server.
* POST - Adds new data to the server.
* PUT - Replaces an existing record
* PATCH - Updates selected fields of a record
* DELETE - Removes a record from the server.

a) Create a JSON Server

Step 1: Initialize the project.

Open the vscode terminal and execute the following commands.

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

Step 2: Create db.json

Create a file named db.json and add the following Student data.

```
{
  "Students": [
    {"id": "1",
     "name": "Simha",
     "course": "CSM",
     "marks": 90
    },
    {"id": "2",
     "name": "Girish",
     "course": "CSE",
     "marks": 90
    },
    {"id": "3",
     "name": "SriNani",
     "course": "IT",
     "marks": 88
    }
  ]
}
```

Step 3: start the JSON server

For the current JSON server version execute

npx json-server db.json

The server start at

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods

a. GET Method: The GET method is used to retreive student records from JSON server.

code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output

b. POST Method: The POST method is used to insert a new student record into the server.

code:

```
fetch("http://localhost:3000/students",
    { method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        name: "Girish",
        course: "CSE"
        marks: 92
      })
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
```

```
    .catch(error => {
        console.log(error);
    });
```

Output:

C. PUT Method: The PUT method is used to replace an existing student record with updated data.

```
code: fetch('http://localhost:3000/student
    { method : PUT
      headers : {
        'Content-Type': 'application/j
      },
      body : JSON.Stringify ({
        name : 'Simha',
        course : 'AIML',
      }) marks : 95
    })
```

```
    .then (response => response.json())
    .then (data => {
        console.log (data);
    })
    .catch (error => {
        console.log (error);
    });
```

output:

D. PATCH Method: The patch method is used to update only selected fields of an existing student record.

```
code: fetch ('http://localhost:3000/students/2
    { method : 'PATCH',
      headers : {
        'Content-Type': 'application/json'
      },
      body : JSON.Stringify ({
        marks : 96
      })
    })
    .then(response => response.json())
    .then (data => {
```

```
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Output:

E. DELETE Method: The DELETE method is used to remove an existing student record from the server.

```
code: fetch('http://localhost:3000/students/1',
    { method: 'DELETE'
  })
  .then(response => {
    if (response.ok) {
      console.log("student deleted successfully");
    }
  })
  .catch(error => {
    console.log(error);
  });
```

Output:

* After executing the POST, PUT, PATCH and DELETE operations in sequence the remaining student will be.

Output:

Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server. The CRUD operations were performed on student records.

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

localhost:3000/students


```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```


2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
}
```


3. POST (Create Student) – Using Postman

localhost:3000/students

localhost:3000/students


```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "_id": "68dcf9b7e4c9e8e1f3a6c5678",
    "__v": 0
}
```


Status: 201 Created Time: 48 ms Size: 160 B

4. PUT (Update Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "AIML",
    "marks": 96,
    "__v": 0
}
```


Status: 200 OK Time: 62 ms Size: 165 B

5. DELETE (Remove Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "message": "Student deleted successfully"
}
```


Status: 200 OK Time: 45 ms Size: 46 B

6. GET All Students (After Deletion)

localhost:3000/students

localhost:3000/students


```
[]
```


Status: 200 OK Time: 28 ms

Experiment - 9

9. Develop CRUD operations using mongoDB.

* Aim To develop a web application using Node.js, Express.js and MongoDB to perform CRUD operation (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON like documents, CRUD operations are used to manage data in a database.

Mangoose is an object data modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations

1. Installation and project Setup

```
mkdir MongoDB.CRUD
cd MongoDB_CRUD
npm init -y
npm install express mongoose
```

2. Program Server.js

```
const express = require ('express');
const mongoose = require ('mongoose');
const app = express ();
const PORT = 3000;
app.use (express.json ())
mongoose.connect ('mongodb://127.0.0.1:
                    27017/college')
  .then (() => console.log ("Mongodb conne
                              -cted"));
  .catch (err => console.log (err));
// create student
const studentSchema = new
mongoose.Schema ({
      name : {
        type : String,
      }, required : true
      course : {
        type : String,
        required : true
      },
      marks : {
        type: number,
        required : true
      }
});
```

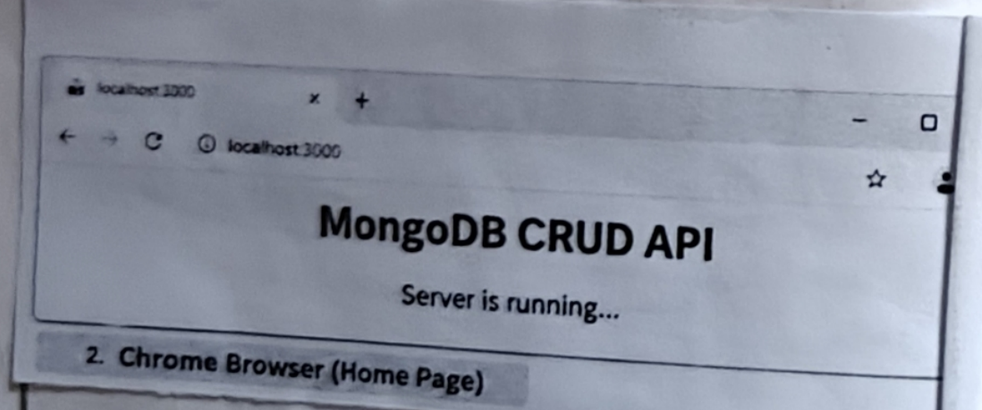
```
// create student model
const student =
    mongoose.model ('student', studentSch
                        -ema);
// create : Add
app.post ('/students', async (req,res) => {
  try {
    const student = new Student (req.bod
                              -y);
    const savedStudent = await
                              student.save();
    res.status (201).json (savedStudent);
  } catch (error) {
    res.status (400).json ({message.
                  error : message});
  }
});
// Read : Get all students
app.get ('/students', async (req,res) => {
  try {
    const students = await Student.find();
    res.status (500).json ({message: error.
                        message});
  }
});
```

```
catch(error){
  res.status(400).json({message :
                        error.message});
  }
});
```

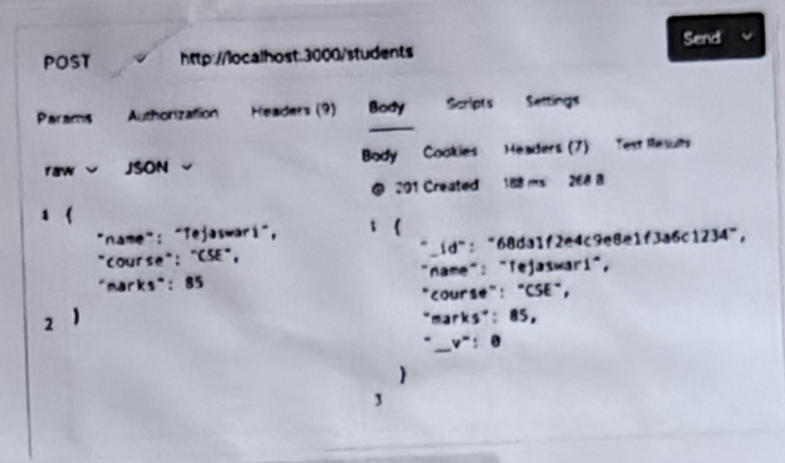
Outputs :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

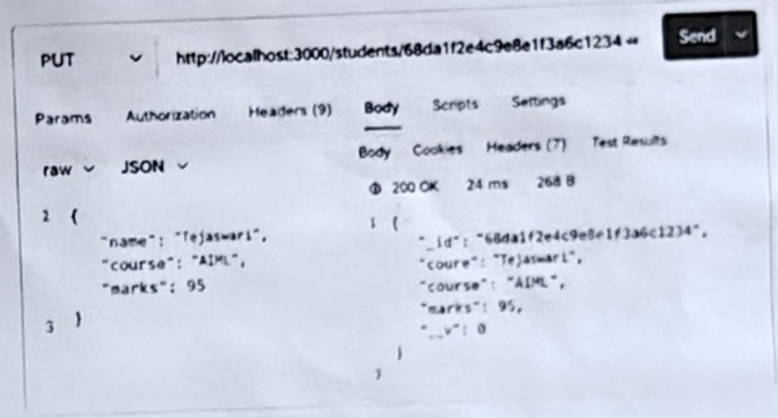
1. [illegible] Output



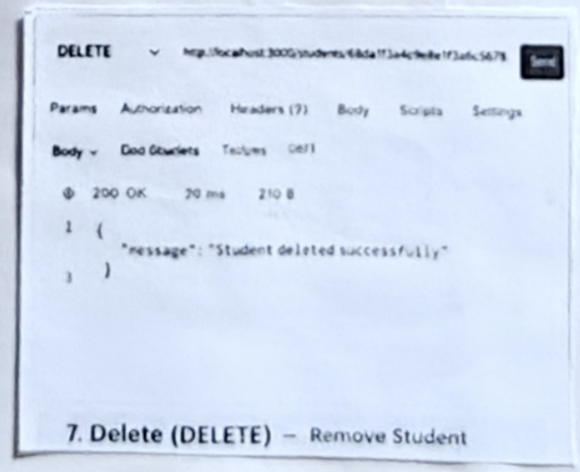
2. Chrome Browser (Home Page)



3. Create (POST) – Add Student



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student

GET http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (7) Body Scripts Settings

rody JSON 200 OK 28 ms 246 B


```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "name": "Tejeswari",
    "course": "CSE",
    "marks": 85,
    "__v": 0
}
```


5. Read (GET) – Get Student by ID

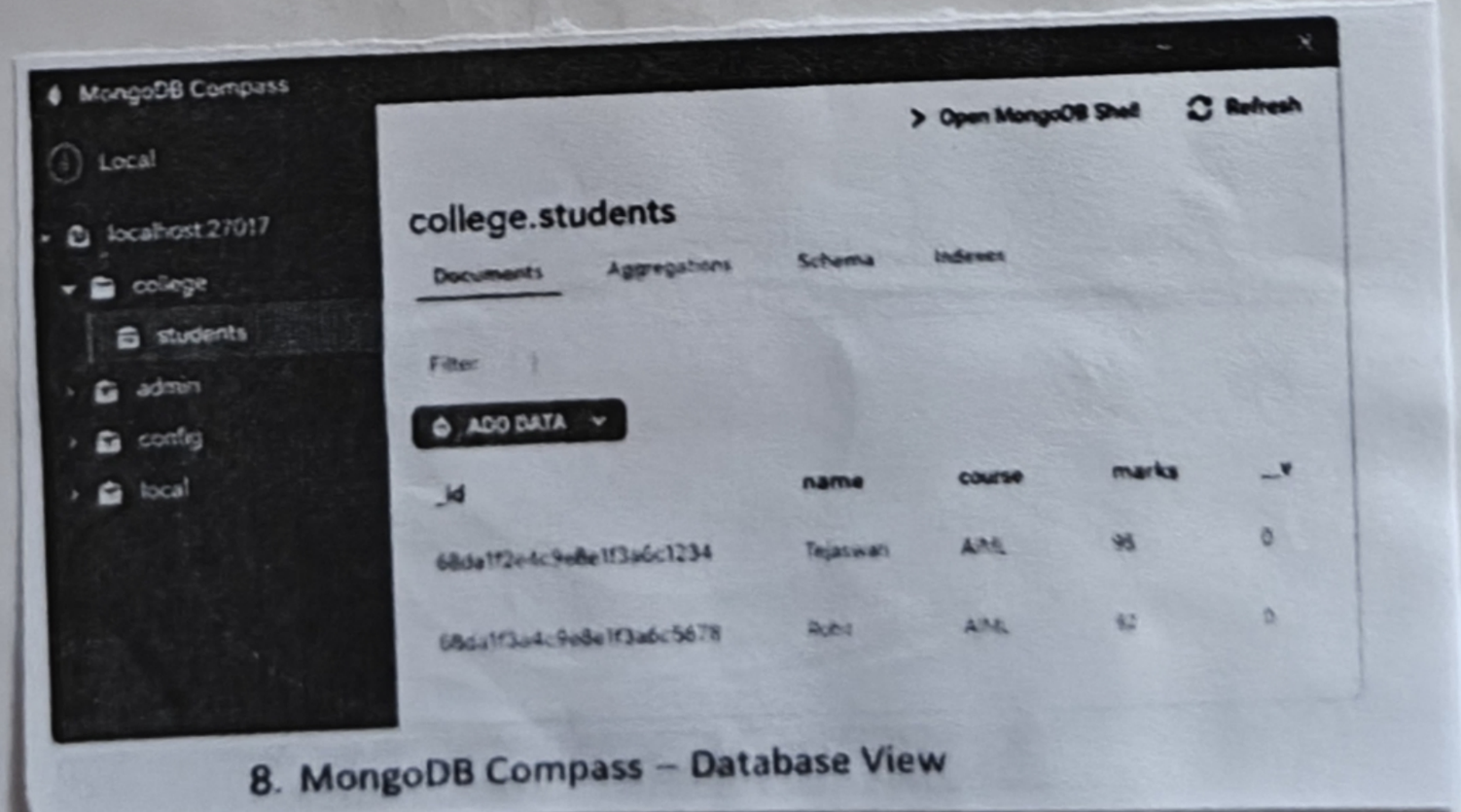
GET http://localhost:3000/students Send

Params Authorization Headers (7) Body Scripts Settings

rody JSON 200 OK 32 ms 512 B


```
[
    {
        "_id": "68da1f2e4c9e8e1f3a6c1234",
        "name": "Tejaswari",
        "course": "CSE",
        "marks": 85,
        "__v": 0
    },
    {
        "_id": "68da1f3a4c9e8e1f3a6c5678",
        "name": "Rohit",
        "course": "AIML",
        "marks": 92,
        "__v": 0
    }
]
```


4. Read (GET) – Get All Students



8. MongoDB Compass – Database View